



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251187
Nama Lengkap	Benedictina Andhika Chinor Jodie Soesila
Minggu ke / Materi	12 / Tipe Data Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

MATERI

Dictionary mirip dengan list, tetapi lebih bersifat umum. Dictionary memiliki anggota yang terdiri dari pasangan kunci:nilai. Kunci harus bersifat unik, tidak boleh ada dua kunci yang sama dalam satu dictionary.

Dictionary pada Python digunakan untuk menyimpan data dalam bentuk pasangan key-value. Dictionary kosong dapat dibuat menggunakan fungsi **dict()** atau tanda kurung kurawal {}.

```
1 eng2sp = dict()
2 print (eng2sp)
```

Data pada dictionary ditambahkan menggunakan tanda kurung siku [] dengan format **dictionary[key] = value**.

```
1 eng2sp['one'] = 'uno'
2 print (eng2sp)
```

Dictionary bisa langsung dibuat beserta isinya.

```
1 eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
2 print (eng2sp)
```

Urutan data dalam dictionary tidak tetap karena elemen diakses menggunakan key, bukan indeks angka.

```
1 print (eng2sp['two'])
```

Jika key yang dicari tidak tersedia, maka akan muncul **KeyError**.

```
1 print (eng2sp['four'])
```

Fungsi **len()** digunakan untuk menghitung jumlah item dalam dictionary, sedangkan operator **in** digunakan untuk memeriksa keberadaan key.

```
len(eng2sp)
```

```
'one' in eng2sp
```

Untuk mengambil seluruh nilai dalam dictionary dapat digunakan method **values()**.

```
vals = list(eng2sp.values())  
'uno' in vals
```

Dictionary memiliki proses pencarian lebih cepat dibandingkan list karena Python menggunakan algoritma Hash Table.

Dictionary sebagai set penghitung (counters)

Terdapat beberapa cara untuk menghitung jumlah kemunculan huruf dalam sebuah string, seperti menggunakan banyak variabel, list, atau dictionary. Dari beberapa metode tersebut, penggunaan dictionary dianggap lebih praktis karena tidak perlu menentukan huruf apa saja yang akan muncul sebelumnya. Setiap huruf akan disimpan sebagai key, sedangkan jumlah kemunculannya menjadi value.

Contoh implementasinya adalah sebagai berikut:

```
word = 'brontosaurus'  
d = dict()  
  
for c in word:  
    if c not in d:  
        d[c] = 1  
    else:  
        d[c] = d[c] + 1  
  
print(d)
```

Program tersebut menghasilkan histogram frekuensi huruf:

```
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

Python juga menyediakan method **get()** untuk menyederhanakan proses tersebut. Metode ini akan mengambil nilai dari suatu key, dan jika key tidak ditemukan maka akan menggunakan nilai default.

```
counts = {'chuck': 1, 'annie': 42, 'jan': 100}

print(counts.get('jan', 0))
print(counts.get('tim', 0))
```

get() membuat kode histogram menjadi lebih ringkas karena tidak memerlukan percabangan **if**.

```
word = 'brontosaurus'
d = dict()

for c in word:
    d[c] = d.get(c, 0) + 1

print(d)
```

Dictionary dan File

Penggunaan dictionary dalam Python untuk menghitung jumlah kemunculan kata pada sebuah file teks. Program dilakukan dengan membaca setiap baris file, memecahnya menjadi beberapa kata menggunakan **split()**, lalu menghitung frekuensi setiap kata menggunakan dictionary. Proses ini menggunakan nested loop.

```
fname = input('Enter the file name: ')

try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()

counts = dict()

for line in fhand:
    words = line.split()

    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1

print(counts)
```

Penulisan `counts[word] += 1` merupakan bentuk singkat dari `counts[word] = counts[word] + 1`. Saat program dijalankan, hasil yang ditampilkan berupa jumlah kemunculan setiap kata dalam file teks menggunakan format dictionary.

Looping dan Dictionary

Pada perulangan **for**, dictionary akan melakukan iterasi berdasarkan **key** yang dimilikinya. Setiap key dapat digunakan untuk mengambil nilai yang sesuai dari dictionary.

Contoh programnya:

```
counts = {'chuck': 1, 'annie': 42, 'jan': 100}

for key in counts:
    print(key, counts[key])
```

Output yang dihasilkan menunjukkan bahwa urutan key pada dictionary tidak selalu tetap atau berurutan.

Perulangan juga dapat digunakan untuk menampilkan data tertentu berdasarkan kondisi. Misalnya, hanya menampilkan nilai yang lebih dari 10.

```
counts = {'chuck': 1, 'annie': 42, 'jan': 100}

for key in counts:
    if counts[key] > 10:
        print(key, counts[key])
```

Program tersebut hanya akan menampilkan pasangan key dan value dengan nilai di atas 10.

Jika ingin menampilkan key secara berurutan alfabet, maka key pada dictionary dapat diubah menjadi list menggunakan method **keys()**, kemudian diurutkan menggunakan **sort()**.

```
counts = {'chuck': 1, 'annie': 42, 'jan': 100}

lst = list(counts.keys())
print(lst)

lst.sort()
```

```
for key in lst:
    print(key, counts[key])
```

Advanced Text Parsing

Pada contoh sebelumnya, file teks yang digunakan tidak memiliki tanda baca. Namun pada bagian ini digunakan file teks dengan tanda baca lengkap sehingga diperlukan proses tambahan sebelum menghitung jumlah kata. Dalam Python, fungsi **split()** hanya memisahkan kata berdasarkan spasi, sehingga kata seperti "soft!" dan "soft" akan dianggap berbeda. Hal yang sama juga berlaku pada huruf kapital, misalnya "Who" dan "who" akan dihitung sebagai dua kata yang berbeda.

Untuk mengatasi hal tersebut, digunakan beberapa method string seperti **lower()**, **translate()**, dan modul **string.punctuation** untuk menghapus tanda baca dan mengubah seluruh huruf menjadi huruf kecil.

Contoh programnya:

```
import string

fname = input('Enter the file name: ')

try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()

counts = dict()

for line in fhand:
    line = line.rstrip()
    line = line.translate(line.maketrans('', '', string.punctuation))
    line = line.lower()
    words = line.split()

    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1

print(counts)
```

Pada program tersebut, method **translate()** digunakan untuk menghapus semua tanda baca, sedangkan **lower()** digunakan agar seluruh kata menjadi huruf kecil sehingga perhitungan kata menjadi lebih akurat. Setelah itu, setiap kata dihitung menggunakan dictionary untuk mengetahui frekuensi kemunculannya dalam file teks.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

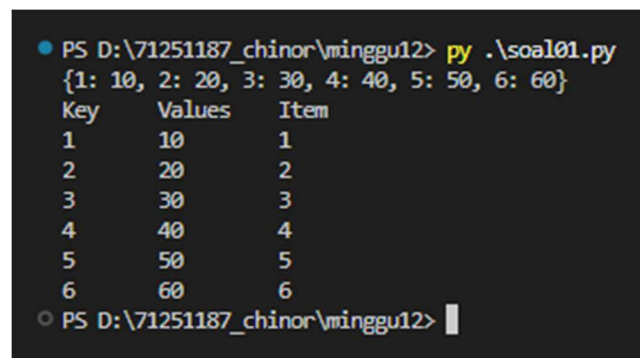
Link Github: https://github.com/chinorjodie/71251187_chinor

SOAL 1

Source Code:

```
def apalah(d):  
    a = 0  
    print("Key      Values      Item")  
    for i, j in d.items():  
        a += 1  
        print(f"{i}      {j}      {a}")  
  
d = dict(eval(input()))  
apalah(d)
```

Output Program:



```
PS D:\71251187_chinor\minggu12> py .\soal01.py  
{1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}  
Key      Values      Item  
1         10         1  
2         20         2  
3         30         3  
4         40         4  
5         50         5  
6         60         6  
PS D:\71251187_chinor\minggu12>
```

Penjelasan:

Kode tersebut digunakan untuk menampilkan isi sebuah dictionary dalam bentuk tabel sederhana yang berisi key, value, dan nomor urut item. Fungsi **apalah(d)** menerima parameter berupa dictionary **d**, kemudian variabel **a** diinisialisasi dengan nilai 0 untuk menghitung jumlah item. Program menampilkan judul kolom menggunakan **print("Key Values Item")**, lalu melakukan

perulangan menggunakan `for i, j in d.items()` untuk mengambil setiap pasangan key dan value pada dictionary. Pada setiap iterasi, nilai `a` akan bertambah 1 menggunakan `a += 1` sebagai nomor urut item, kemudian key, value, dan nomor item ditampilkan menggunakan f-string. Selanjutnya, program meminta input dictionary dari pengguna melalui `input()`, lalu `eval()` digunakan untuk mengubah input menjadi objek Python dan `dict()` memastikan data berbentuk dictionary. Setelah itu, fungsi `apalah(d)` dipanggil untuk menampilkan seluruh isi dictionary beserta nomor itemnya.

SOAL 2

Source Code:

```
def menggabung(a,b):
    nyaw = {}
    if len(a) == len(b):
        for i in range(len(a)):
            nyaw[a[i]] = b[i]
        print(nyaw)
    else:
        print("List Harus Sama Panjang!")

list1 = eval(input())
list2 = eval(input())
menggabung(list1,list2)
```

Output:

```
● PS D:\71251187_chinor\minggu12> py .\soal02.py
['red', 'green', 'blue']
['#FF0000', '#008000', '#0000FF']
{'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
○ PS D:\71251187_chinor\minggu12> █
```

Penjelasan:

Kode tersebut digunakan untuk menggabungkan dua buah list menjadi sebuah dictionary. Fungsi **menggabung(a,b)** menerima dua parameter berupa list, yaitu **a** sebagai key dan **b** sebagai value. Di dalam fungsi dibuat dictionary kosong bernama **nyaw**. Program kemudian memeriksa apakah panjang kedua list sama menggunakan **if len(a) == len(b)**. Jika panjangnya sama, dilakukan perulangan menggunakan **for i in range(len(a))** untuk mengambil elemen dari kedua list berdasarkan indeks yang sama, kemudian elemen dari list **a** dijadikan key dan elemen dari list **b** dijadikan value pada dictionary **nyaw**. Setelah seluruh data dimasukkan, dictionary akan ditampilkan menggunakan **print(nyaw)**. Namun, jika panjang kedua list berbeda, program akan menampilkan pesan **"List Harus Sama Panjang!"** karena setiap key harus memiliki pasangan value. Selanjutnya, program meminta input dua list dari pengguna menggunakan **eval(input())**, lalu kedua list tersebut dikirim ke fungsi **menggabung()** untuk diproses menjadi dictionary.

SOAL 3

Source Code:

```
import re

def cariwoi(file):
    m = {}
    for l in file:
        emel = re.findall(r"From (\b\S+@\S+\b)", l)
        for i in emel:
            m.update({i : m.get(i, 0) + 1})

    print(m)
fname = input("Nama File : ")
file = open(fname)
cariwoi(file)
```

Output:

```
PS D:\71251187_chinor\minggu12> py .\soal03.py
Nama File : mbox-short.txt
{'stephen.marquard@uct.ac.za': 2, 'louis@media.berkeley.edu': 3, 'zqian@umich.edu': 4, 'rjlowe@iupui.edu': 2, 'cwen@iupui.edu': 5, 'gsilver@umich.edu': 3, 'wagnermr@iupui.edu': 1, 'antranig@caret.cam.ac.uk': 1, 'gopal.ramasammycook@gmail.com': 1, 'david.horwitz@uct.ac.za': 4, 'ray@media.berkeley.edu': 1}
PS D:\71251187_chinor\minggu12> █
```

Penjelasan:

Kode tersebut digunakan untuk mencari dan menghitung jumlah kemunculan alamat email pada sebuah file teks menggunakan dictionary dan regular expression (*regex*). Modul **re** diimpor terlebih dahulu untuk menggunakan fungsi pencarian pola teks. Fungsi **cariwai(file)** menerima parameter berupa file yang akan dibaca, kemudian dibuat dictionary kosong **m = {}** untuk menyimpan email sebagai key dan jumlah kemunculannya sebagai value. Program melakukan perulangan **for l in file** untuk membaca file baris per baris. Pada setiap baris, fungsi **re.findall(r"From (\b\S+@\S+\b)", l)** digunakan untuk mencari alamat email yang diawali kata "From". Hasil pencarian disimpan dalam variabel **emel**. Selanjutnya dilakukan perulangan **for i in emel** untuk memproses setiap email yang ditemukan. Method **m.get(i, 0) + 1** digunakan untuk menambah jumlah kemunculan email; jika email belum ada di dictionary maka akan diberi nilai awal 0 kemudian ditambah 1. Fungsi **update()** digunakan untuk memperbarui isi dictionary dengan jumlah terbaru. Setelah seluruh file selesai diproses, dictionary **m** akan ditampilkan menggunakan **print(m)**. Pada bagian akhir program, pengguna diminta memasukkan nama file melalui **input()**, kemudian file dibuka menggunakan **open(fname)** dan dikirim ke fungsi **cariwai(file)** untuk diproses.

SOAL 4

Source Code:

```
import re

def carilagi(file):
    m = {}
    for l in file:
        em = re.findall(r"From \b\S+@\S+\b)", l)
        for i in em:
            m.update({i : m.get(i, 0) + 1})

    print(m)

fname = input("Nama File : ")
```

```
file = open(fname)
carilagi(file)
```

Output:

```
• PS D:\71251187_chinor\minggu12> py .\soal04.py
  Nama File : mbox-short.txt
  {'uct.ac.za': 6, 'media.berkeley.edu': 4, 'umich.edu': 7, 'iupui.edu': 8, 'caret.cam.ac.uk': 1, 'gmail.com': 1}
○ PS D:\71251187_chinor\minggu12> █
```

Penjelasan:

Kode tersebut digunakan untuk mencari dan menghitung jumlah kemunculan domain email pada sebuah file teks menggunakan dictionary dan regular expression (*regex*). Modul **re** diimpor terlebih dahulu agar program dapat melakukan pencarian pola teks. Fungsi **carilagi(file)** menerima parameter berupa file yang akan dibaca, kemudian dibuat dictionary kosong **m = {}** untuk menyimpan domain email sebagai key dan jumlah kemunculannya sebagai value. Program membaca file baris per baris menggunakan perulangan **for l in file**. Pada setiap baris, fungsi **re.findall(r"From \b\S+@(\S+\b)", l)** digunakan untuk mengambil bagian domain dari alamat email yang diawali kata "From". Tanda kurung **()** pada regex digunakan untuk mengambil hanya bagian setelah simbol **@**, misalnya **gmail.com**. Hasil pencarian disimpan pada variabel **em**. Selanjutnya dilakukan perulangan **for i in em** untuk memproses setiap domain yang ditemukan. Method **m.get(i, 0) + 1** digunakan untuk menghitung jumlah kemunculan domain, sedangkan **update()** digunakan untuk memperbarui isi dictionary. Setelah seluruh file selesai diproses, dictionary berisi daftar domain dan jumlah kemunculannya akan ditampilkan menggunakan **print(m)**. Pada bagian akhir program, pengguna diminta memasukkan nama file melalui **input()**, kemudian file dibuka dengan **open(fname)** dan dikirim ke fungsi **carilagi(file)** untuk diproses.